SHOPINGY - Vendor Brief: Architecture Overview & Web Scraping Framework

Client: Shopingy s.r.o. | Date: 2026-03-25

Confidential - no NDA in place. Integration-level detail only. Do not redistribute.

1. Company Context

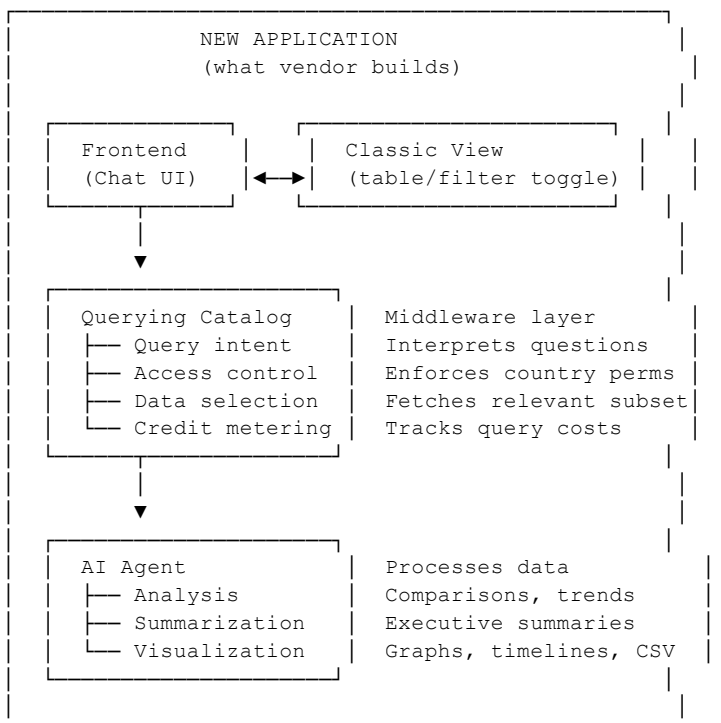
SHOPINGY is a B2B SaaS platform providing retail market intelligence across 40+ countries. The platform tracks tenant changes in 7,700+ shopping malls with 340,000+ tenant records and ~5 years of historical data.

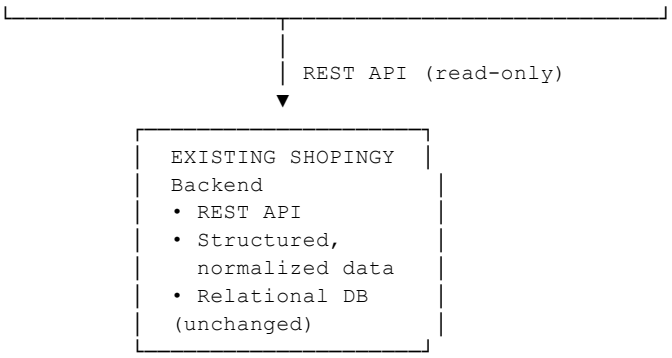
This document provides the technical context required for vendors to design and scope the AI transformation project described in the accompanying Functional Requirements (Wishlist) document.

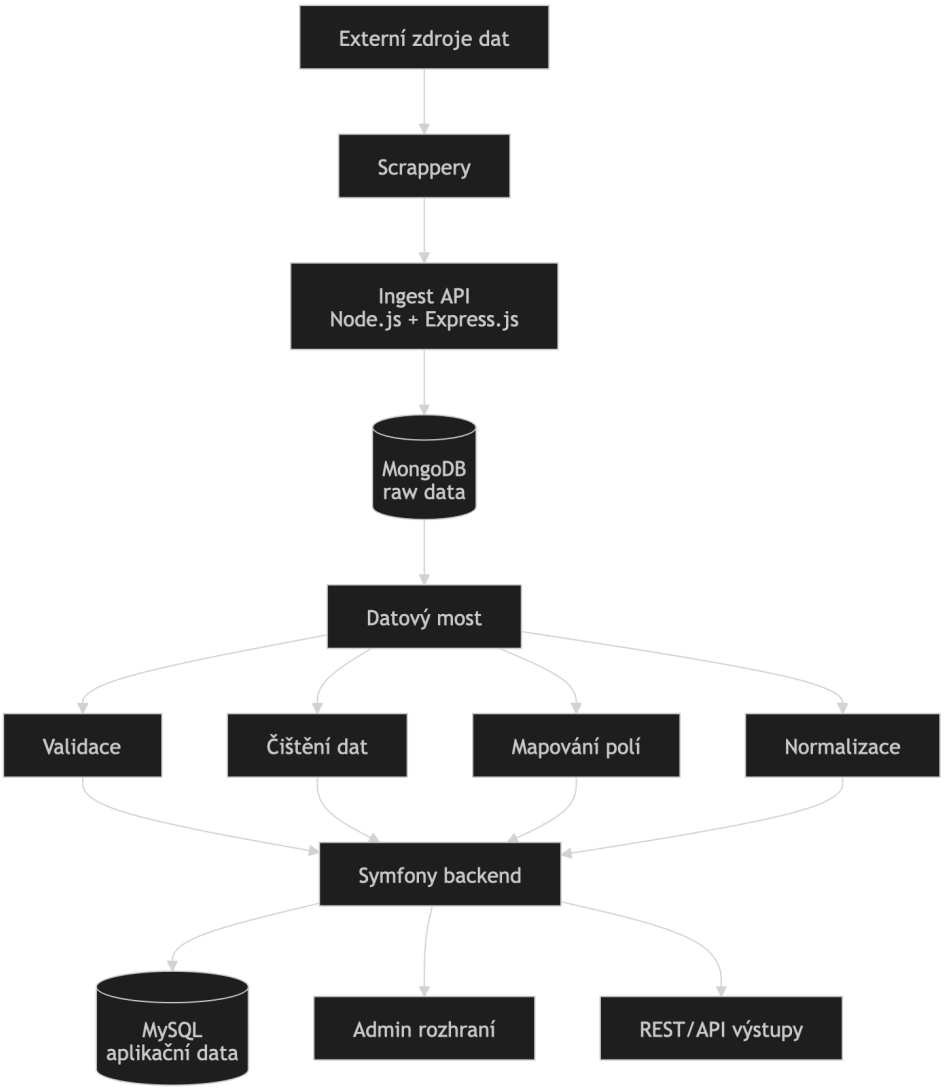
2. Integration Architecture

2.1 What the Vendor Connects To

The SHOPINGY backend is an existing, stable system. The vendor builds the new AI application as a separate service that connects to the existing backend via REST API.







2.2 What the Backend Provides

Aspect: API type
Detail: REST API endpoints

Aspect: Access
Detail: Read-only

Aspect: Data format
Detail: Structured, cleaned, and normalised relational data

Aspect: Data content
Detail: Malls, tenants, brands, locations, historical changes (~5 years)

Aspect: Data freshness
Detail: Updated monthly

Aspect: Scale
Detail: 7,700+ malls, 340,000+ tenants, 1,340+ brands, 40+ countries

Aspect: Normalization
Detail: Tenant/brand names already normalized to canonical forms (e.g., all variants of "H&M" mapped to one identifier)

Aspect: Unique identifiers
Detail: Each mall has a unique ID - use this as the primary key for all queries

2.3 What the Vendor Does NOT Touch

• The existing backend, database, and data pipeline remain unchanged
• No direct database access - all data access goes through the REST API
• No write operations to the existing system
• Detailed API documentation, database schema, and internal architecture will be shared after vendor selection and NDA signing

3. Querying Catalogue - Middleware Layer

The Querying Catalogue is the core middleware between the user interface and the backend. It acts as a controlled intermediary:

1. Receives the user's natural language query from the frontend
2. Interprets query intent (lookup, comparison, trend, export, summary)
3. Checks user's geographic access permissions (country-level)
4. Determines which data subset is needed to answer the query
5. Fetches only the relevant data from the backend via REST API
6. Passes data to the AI agent for analysis and output generation
7. Measures the query cost (credits based on complexity and data volume)

Why this approach: The AI agent never gets full database access. This prevents data leakage, enforces access control at the data level, and enables credit-based pricing per query.

4. Security Architecture

4.1 Security Requirements

Concern: SQL injection prevention
Required Approach: AI-generated queries must be fully sanitized and parameterized. No raw SQL passthrough from user input to database. The Querying Catalogue must validate all queries before execution.

Concern: Data access enforcement
Required Approach: Country-level permissions enforced at the Catalogue layer - the AI agent cannot bypass geographic restrictions

Concern: User data protection
Required Approach: Users cannot access, extract, or manipulate data beyond their subscription scope

Concern: Encryption
Required Approach: At rest and in transit

Concern: Audit trail
Required Approach: All queries, exports, and access events logged

4.2 Questions for the Vendor (from our technical team)

These questions were raised during internal architecture review and must be addressed in your proposal:

1. How will the AI agent communicate with the database safely? Specifically, how do you prevent SQL injection or other injection attacks when the AI translates natural language into data queries? We need a clear explanation of the query sanitisation approach.

2. How do you ensure the AI does not return inaccurate or hallucinated data? The AI must be grounded in actual database records. When a user asks a complex question, how does the system verify that the response is factually correct and based on real data? What safeguards prevent the AI from generating plausible but incorrect answers?

3. How do you handle data processing requirements for different query types? Some queries are simple lookups (one country, one brand), others require multi-country aggregation with historical analysis. How does your architecture handle this range of complexity efficiently? What are the expected response times?

4. How do you prevent bulk data extraction? The credit system must prevent users from extracting the entire database through repeated simple queries. What rate-limiting, credit pricing, or other mechanisms do you propose?

5. Web Scraping Framework

5.1 Overview

The data in the SHOPINGY database is collected via automated web scraping from public sources.

Parameter: Data sources
Detail: Public shopping mall websites (tenant directories, sitemaps)

Parameter: Scraper count
Detail: Thousands of automated scrapers

Parameter: Frequency
Detail: Monthly cycle per mall

Parameter: Coverage
Detail: 7,700+ malls across 40+ countries, actively growing

Parameter: Processing
Detail: Automated collection + manual verification + data normalisation

Parameter: Data legality
Detail: Public sources only - no proprietary data, no licensing restrictions

5.2 How Data Reaches the API

1. Automated scrapers collect tenant data from shopping mall websites monthly
2. Raw data goes through a cleaning and normalization pipeline
3. Cleaned, structured data is stored in the application database
4. The REST API serves this clean data to the frontend and (in the future) to the new AI application

The scraping pipeline is separate from the AI project scope. The vendor does not build, modify, or interact with the scraping system. The vendor's application reads already-cleaned, normalized data via the REST API.

5.3 Data Normalization

The biggest data challenge: mall operators name tenants inconsistently across countries and languages. The same brand may appear differently on different mall websites. SHOPINGY's pipeline normalizes all variants to a canonical brand identifier, making cross-country comparison possible.

The AI layer works with already-normalized data - no additional normalization is needed from the vendor.

5.4 Key Data Characteristics for the AI Layer

• Historical depth: ~5 years of tenant change records - enables trend analysis
• Monthly freshness: Data is updated monthly. The AI should communicate this to users (e.g., "Data as of March 2026").
• Growing database: New malls and countries are added regularly. The application must scale with growing data volumes.
• Unique mall IDs: Each mall has a unique identifier. This should be the anchor for all queries, not GPS or text-based name matching.

6. Technical Constraints & Boundaries

Constraint: No direct database access
Detail: Connect via REST API only

Constraint: Read-only
Detail: No writes to existing SHOPINGY database

Constraint: No NDA in place
Detail: This document shares integration-level information only. Detailed schema, API documentation, and internal architecture shared after vendor selection + NDA signing.

Constraint: Existing product stays live
Detail: Classic table/filter UI must remain accessible as fallback in the new application

Constraint: Country-level access enforced
Detail: AI cannot bypass geographic restrictions under any circumstances

Constraint: No vendor lock-in
Detail: Solution must be portable - no proprietary platforms or mandatory subscriptions tying SHOPINGY to a single vendor

Constraint: LLM model flexibility
Detail: Architecture must allow switching between LLM providers without major rework

7. What We Need in Your Proposal

1. Proposed technology stack - frontend, middleware/catalog, AI model, hosting
2. Architecture diagram - how your solution connects to the existing SHOPINGY backend via REST API
3. Querying Catalog design - how you implement query interpretation, access control, and selective data retrieval

4. AI model choice and reasoning - cloud API vs. self-hosted vs. fine-tuned? How would you handle model switching?
5. Credit system design - how you meter query costs and prevent bulk extraction
6. Security strategy - detailed answer to the security questions in Section 4.2, especially SQL injection prevention and data access enforcement
8. Data accuracy approach - how you prevent AI hallucination and ensure responses are grounded in real data
9. Timeline and milestones - phased delivery plan
10. Cost breakdown - development, infrastructure, ongoing operational costs
11. Team and experience - who works on the project, relevant AI/data product experience
12. Vendor lock-in statement - confirm solution portability and no mandatory proprietary dependencies

For additional questions, please contact:
Martin Landsmann, AI Product Manager (martin@rousing-ai.com)
Hana Kabourková, Global Business Development Director (kabourkova@shopingy.com)